Sustained Telegram flood-wait pressure from unbounded channel polling

Incident report · TScopier listener · 2026-08-24

SEVERITY	Medium	STATUS	Open — fix implemented
DATES	Observed 2026-08-23 → 2026-08-24 (ongoing)	AFFECTED USERS	c8a32918…ccb1 (22 channels); all others unaffected or isolated
COMPONENT	`worker/src/userListener.ts` (Telegram polling)	ROOT CAUSE	Fast-poll graduation defect + no per-account poll rate cap

1. Executive summary (plain English)

One user copies 22 Telegram channels, nearly twice as many as any other customer. For channels where Telegram does not push messages to us immediately, our system fetches the messages itself every 3 seconds. A defect prevents those channels from ever leaving this fast fetching cycle, because a successful fetch does not count as proof that the channel is working. As a result the system sends up to roughly 440 read requests per minute on this one account, and Telegram answers most of them with "slow down" instructions. The listener stays stable thanks to the August 21 backoff fix, but the request volume itself remains far too high. Two targeted corrections to the polling schedule remove the excess volume without touching trade execution or signal handling.

2. Issue encountered

- Railway production logs show **100–300 aggregated flood waits per minute**, with individual waits of 13–26 seconds.
- The adaptive backoff deployed on 2026-08-21 works as designed: pauses escalate **120s → 240s → 480s**; there are no crashes and no reconnect storms.
- Despite the pauses, throttling resumes at full volume after each pause expires — a repeating cycle rather than recovery.
- The affected account is responsible for nearly all recorded flood waits; only one other account (11 channels) triggered a single backoff event.

3. Affected user(s)

USER	ACTIVE CHANNELS	FLOOD-WAIT PATTERN	SIGNALS PROCESSED (24H)
`c8a32918-9d96-4478-9869-a9e9cb1eccb1`	**22**	100–300 waits/min; backoffs 120s–480s	~80 rows across all channels
All other users	≤ 13	None or isolated single events	Normal

Channel-count distribution across users: 22, 13, 12, 11, 10, 8, 8, 6, 6, 5 … The affected user is a clear outlier.

4. Root cause

In plain English: Our system receives channel messages in two ways. Normally Telegram delivers each new message to us immediately, at no cost. When Telegram stops delivering messages for a channel, our system asks Telegram for that channel's latest messages instead — and it repeats this question every 3 seconds. The system was designed to notice when a channel is being delivered normally and stop asking; but the test it uses for "this channel is fine" can only be passed by delivered messages, never by answered questions. So a poll that succeeds still leaves the channel marked as "not delivering", and the system keeps asking every 3 seconds forever. On top of that, there is no limit on how many channels can be asked per cycle and no adjustment based on how many channels a user has. This user copies 22 channels; all of them are stuck in the fast cycle. That produces up to about 440 requests per minute on one Telegram account, and Telegram answers most of them with a wait instruction.

Technical detail: Two independent defects combine:

1. **Graduation defect** (`worker/src/userListener.ts`). The function `bumpLastLive` (:3652) records the time of the last live-delivered message and is called only from the live event handlers (:1799, :1830). When the fast poll itself successfully retrieves new messages, only `bumpLastSeen` is called; `last_live_at` is never updated. The fast-poll staleness check (`runFastPoll` , :4674; check at :4696) requires a recent `last_live_at` , so any channel for which Telegram does not deliver live updates fails the check on every cycle indefinitely, even while polling succeeds.
2. **No rate cap.** Each 3-second tick polls every stale channel (`FAST_POLL_INTERVAL_MS` , :165), with no limit on batch size and no adjustment for total channel count. With 22 stuck channels this yields up to ~440 read requests per minute on one Telegram account, far beyond Telegram's limits.

Evidence: a production database snapshot shows all 22 channels of the affected user with null or stale `last_live_at` , while several show fresh `last_seen_at` values written by polling itself — proof that polling succeeds but cannot reclassify a channel as active.

5. The fix (proposed)

Both changes are confined to the polling schedule inside `worker/src/userListener.ts` .

1. **Count successful polls toward liveness.** After a poll completes without errors, record an in-memory marker that extends the effective liveness window for that channel. Channels are polled progressively less often (3s growing toward 15–30s) instead of being re-polled at maximum frequency forever. Channels that error continue into the existing invalid-channel auto-disable path unchanged.
2. **Cap the batch size per tick.** Poll at most approximately six channels per 3-second cycle, always selecting the channels checked longest ago (least-recently-polled order). Every channel retains coverage within roughly 11 seconds regardless of how many channels a user has, and total request volume becomes bounded by arithmetic.

Safety of the change: trade execution, signal parsing, duplicate protection, the adaptive backoff, and the invalid-channel auto-disable path are not modified. Worst case for either change is a few extra seconds of delay before a rarely-active channel's message is noticed — whereas today, during throttling, delays are measured in tens of seconds and during backoff pauses nothing is read at all, so real-world latency improves.

6. Files changed

Not yet implemented. Planned scope:

- `worker/src/userListener.ts` — liveness marking in the poll success path; batch cap and rotation ordering in `runFastPoll` .
- New tests alongside existing suites (node:test) covering graduation behavior, batch cap, and rotation fairness.

7. Verification

- `npm --prefix worker run build` (typecheck).
- Targeted node:test suites for the changed logic plus the full worker test suite (`npm --prefix worker test`).
- Deploy to staging first; confirm on Railway logs that aggregate flood waits fall substantially and signal pickup latency stays within normal bounds before promoting to production.

8. Deployment status

Investigation complete 2026-08-24. No code has been changed yet; nothing deployed. Current production continues to rely on the August 21 adaptive backoff, which keeps the process stable but does not reduce request volume.

9. Follow-ups

1. Implement fixes 1 and 2 above with tests; deploy staging-first per standard practice.
2. Confirm on production logs within 24 hours of deployment that aggregated flood waits for the affected account drop to near zero and no new signal-latency issues appear.
3. Consider whether the channel-add flow should warn users copying very large numbers of channels about expected pickup latency.
4. Unrelated observation from the same log review: Cerebras parser keys are returning HTTP 402 and falling back to OpenAI repeatedly — quota should be checked separately.